

红黑树：基于二叉排序树，表达并实现2-3-4树(2-3树)的数据结构

3阶或4阶B树 转化为红黑树：按序拆分 添加颜色

内部终端结点：没有孩子的结点

叶子结点：失败结点，空结点

性质：结点有颜色 黑色或红色

1.根结点一定是黑色的

2.叶子结点一定是黑色的

3.不存在相邻的两个红色结点（即 红色结点的父亲和孩子一定是黑色的）

4.对于每个结点，从该结点到其下面任意一个叶子结点的简单路径上，所含的黑色结点的数目相同。---->因为B树绝对平衡

5.顺序性。

性质：左根右 根叶黑 不红红 黑路同

结点的黑高 bh：从某个结点出发，到其下一个叶子结点的简单路径上的黑色结点的数目(起点不算)

树的黑高：根结点的黑高

红黑树的黑高 是其对应的B树的高度。

1.平衡性：高的子树 不超过 矮的子树高度的2倍

从根结点到叶子结点的**最长路径长度 不大于最短路径长度的两倍。**

从根结点到叶子结点的**最短的路径长度：黑高bh**

从根结点到叶子结点的**最长的路径长度：2*bh**

2.n个内部结点的红黑树 高度 $h \leq 2\log(n+1)$

红黑树的高度为h 黑高为bh $bh \leq h \leq 2bh \rightarrow bh \geq h/2$

黑高为bh的红黑树 结点个数n，当该红黑树没有红色结点时 结点个数最少， 该红黑树就是一个高度为bh的满二叉树， $n \geq 2^{bh-1}$

黑高为bh的红黑树 结点个数 $n \geq 2^{bh-1}$

$n+1 \geq 2^{bh}$

$\log(n+1) \geq bh \geq h/2$

$h/2 \leq \log(n+1) \rightarrow h \leq 2\log(n+1)$

红黑树的应用：

AVL：平衡性限制的太严格。不适用于动态查找

红黑树：平衡性的限制放宽了 。相较于avl树 更适合动态查找

1.map set /TreeMap TreeSet

2.Linux CPU调度---完全公平调度

3. Linux epoll多路复用

操作：插入。删除

RBT的插入操作：一棵红黑树上插入结点z

性质：左根右 根叶黑 不红红 黑路同

1.新建一个结点z 把待插入的数据放到z中，按照BST插入过程，把z插入到红黑树中。---->满足了顺序性

2.结点z的初始颜色：红色

（1）黑色：一定破坏“黑路同”，每次插入之后都得进行调整

（2）红色：该结点做叶子结点时破坏“根叶黑”，该结点的父亲也是红色时破坏“不红红”，其他情况不破坏性质

3.是否破坏性质，如何调整

（1）z是红色，且是空树插入z，z做根结点，此时破坏“根叶黑”-----直接把z赋值为黑色

（2）z是红色，插入后，z的父亲也是红色---->见下面的分类讨论

破坏不红红 具体的分类及其调整

新插入的结点z 红色的

z的父亲结点p 红色的

z的爷爷结点g 黑色的

z的叔叔结点u u为空结点是黑色的/u存在时 一定是红色---->随着调整的进行，如果向上循环调整了，新的z的树叔叔叔结点u存在时可能红 可能黑

一：叔叔是红色的

p和u变黑。g变红。如果g的父亲是黑的，调整结束。如果g的父亲是红的，g及其父亲又破坏不红红--->继续向上循环调整：把g看成z，把g的父亲看成p 再继续判断满足哪种调整情况。如果g一直是红的 那么当g为根结点时 就可以结束了，把根结点变黑。

二。叔叔是黑色的(因为涉及循环向上调整，后面的调整过程中，如果叔叔是黑色 叔叔是不一定为null的)

调整过程中不涉及到u，不需要考虑u了

仿照AVL树的插入失衡的4种情况，针对p和z的方向分4类

RR：p是g的右孩子 z是p的右孩子

先对g左旋，g变红，p变黑。

RL：p是g的右孩子 z是p的左孩子

先对p右旋，再对g左旋，g变红，z变黑

LL：先对g右旋，g变红，p变黑。

LR：先对p左旋，再对g右旋，g变红，z变黑

